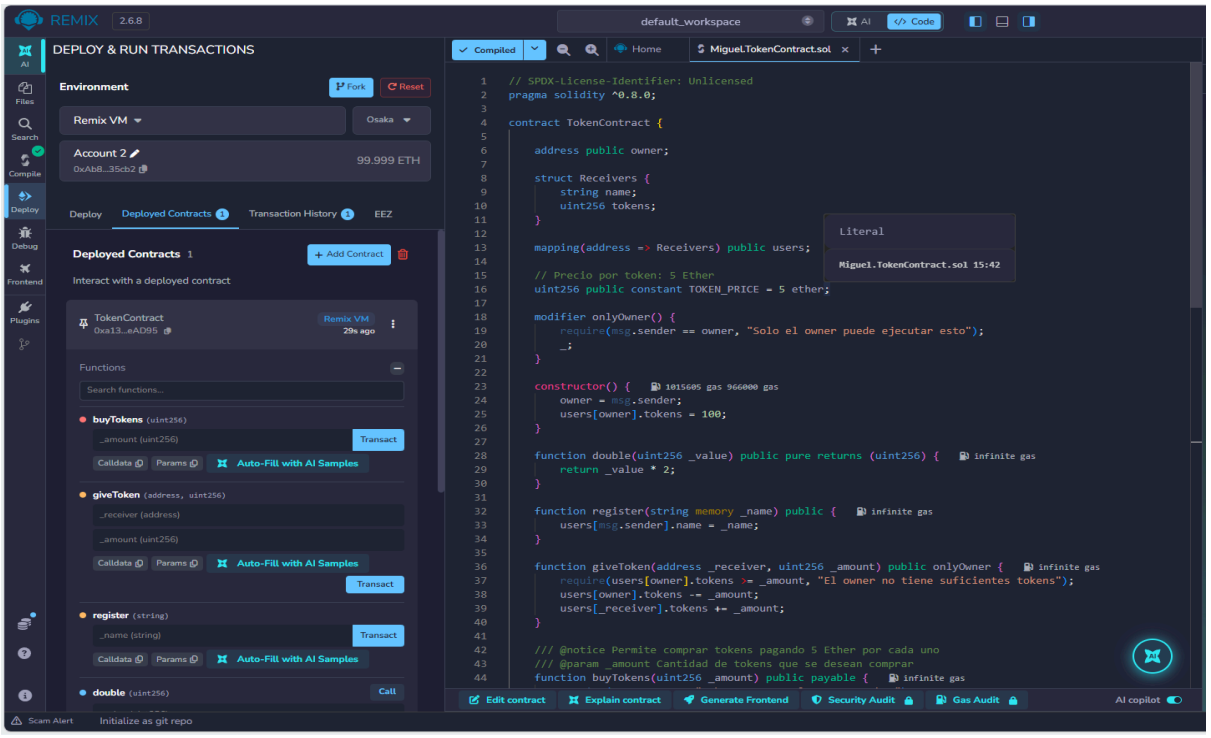
Implementación y pruebas del contrato inteligente TokenContract.sol en Solidity para habilitar la compra de tokens por Ether a un precio establecido de 1 token = 5 ETH, integrando control de inventario, manejo seguro de excedentes y consulta de fondos.

Funciones implementadas

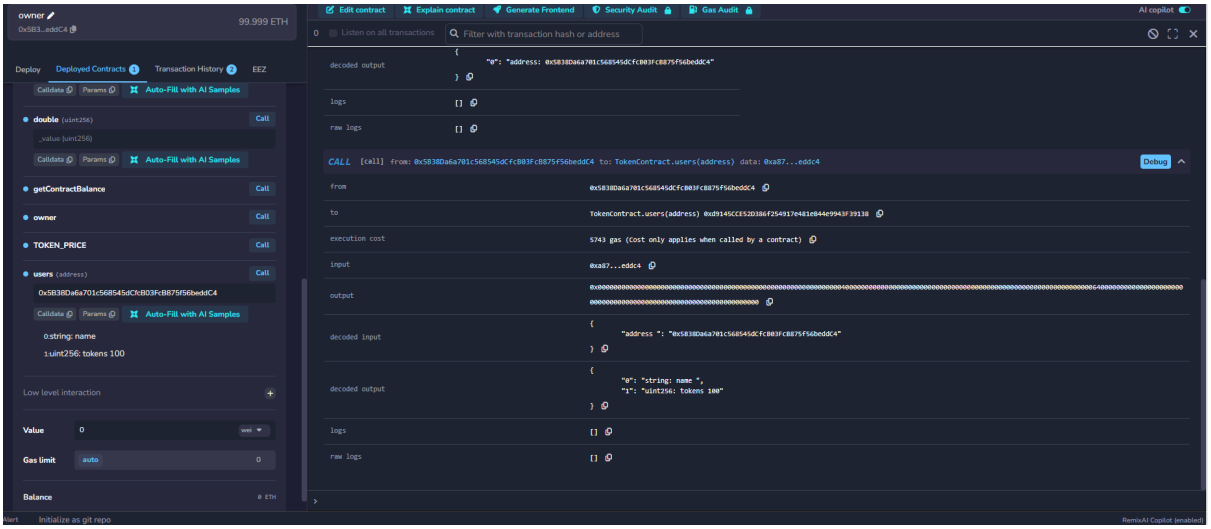
- Resolución de compatibilidad del compilador: Se ajustó la directiva pragma solidity para sincronizar el código con la versión del compilador seleccionada en Remix IDE y evitar errores de análisis sintáctico.
- buyTokens(uint256 _amount): Función pública y payable que permite a cualquier dirección externa adquirir una cantidad específica de tokens.
    - Cálculo de coste: Se fijó la constante de precio TOKEN_PRICE = 5 ether y se computó el total requerido multiplicando la cantidad solicitada por dicho valor.
    - Validaciones mediante require:
        - Comprueba que la cantidad a comprar sea mayor que cero.
        - Valida que el Ether adjunto en la llamada (msg.value) cubra el coste total de la operación.
        - Verifica que la cuenta del propietario (owner) cuente con existencias suficientes en users[owner].tokens.
    - Transferencia interna de saldo: Descuenta los tokens adquiridos del balance de owner y los acredita al comprador (msg.sender) dentro del mapping users.
    - Devolución de excedentes segura: En caso de que el comprador envíe más Ether del necesario, el contrato calcula la diferencia y la reintegra mediante el método de bajo nivel call{value: refundAmount}("") en sustitución del obsoleto .transfer().
- getContractBalance(): Función de lectura (view) que retorna el balance total de Ether resguardado dentro de la dirección del propio contrato inteligente (address(this).balance).

Pruebas en Remix IDE

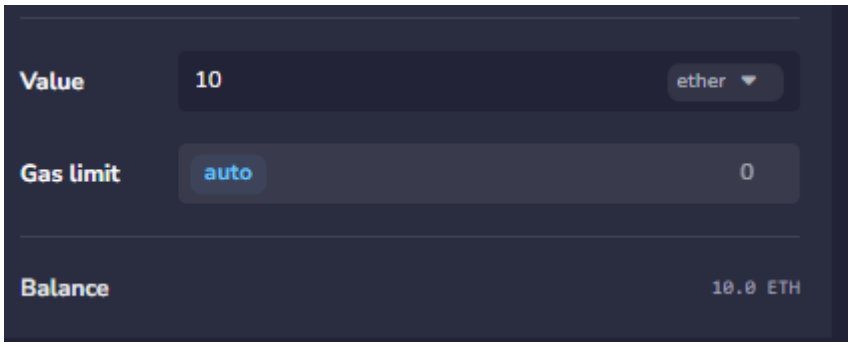
1. **Compilación:** Se compiló el contrato en la pestaña Solidity Compiler garantizando cero advertencias y cero errores de compatibilidad.

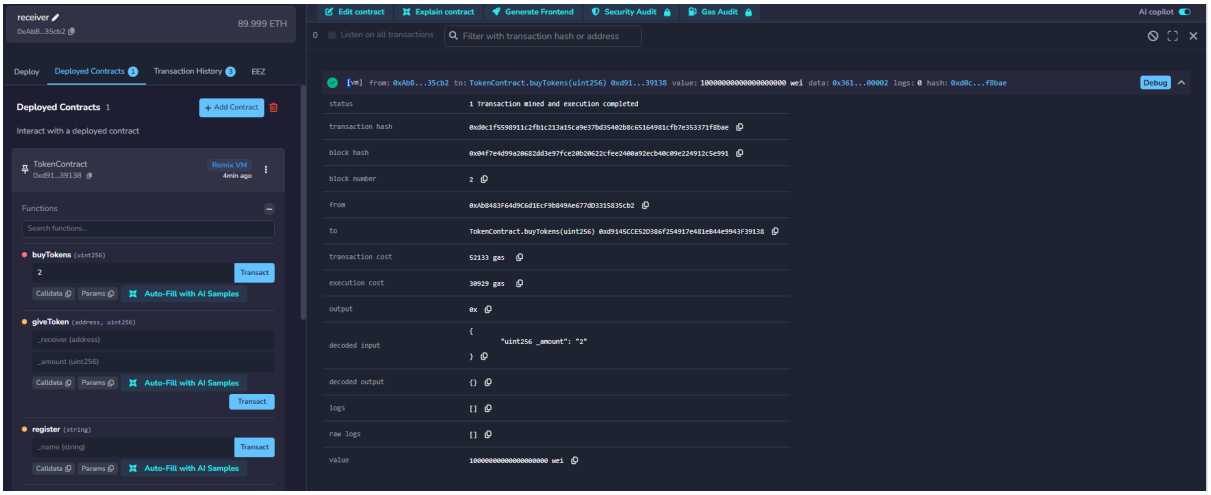


2. **Despliegue:**
   - Se utilizó el entorno simulado Remix VM.
   - Se desplegó el contrato desde la primera cuenta (0x5B3...), la cual quedó registrada automáticamente como owner con una asignación inicial de 100 tokens.



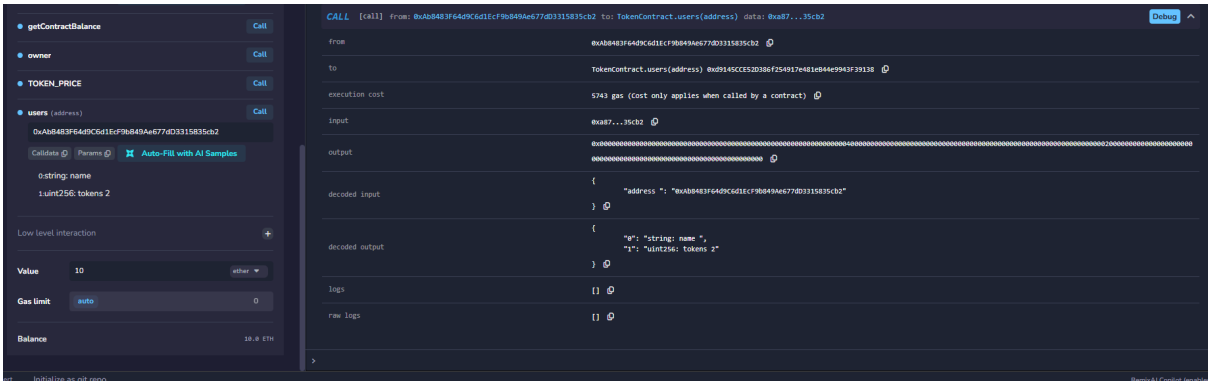
3. **Simulación de compra:**
    - Se cambió la cuenta activa en Remix a una segunda dirección para actuar como cliente externo.
    - Se configuró el campo Value en 10 Ether y se ejecutó la función buyTokens pasando el argumento _amount = 2.





4. **Verificación de estados:**
    - Se consultó la estructura users con la dirección del comprador, comprobando que sus tokens se incrementaron a 2.
    - Se verificó la reducción del saldo de tokens en la dirección del owner (de 100 a 98).
    - Se invocó la función getContractBalance(), confirmando el almacenamiento exitoso de 10000000000000000000 wei (10 ETH) en el contrato.





5. **Comprobación de errores:**
    - Se validó el rechazo de compras cuando el Ether enviado era inferior al total requerido.
    - Se comprobó el bloqueo de la transacción cuando la cantidad de tokens solicitada excedía el stock disponible del propietario.




```
transact to TokenContract.buyTokens errored: Error occurred: Error occurred: revert.

revert
        The transaction has been reverted to the initial state.
Reason provided by the contract: "Ether insuficiente: 1 token cuesta 5 Ether".
If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.

revert
        The transaction has been reverted to the initial state.
Reason provided by the contract: "Ether insuficiente: 1 token cuesta 5 Ether".
If the transaction failed for not having enough gas, try increasing the gas limit gently.

If the transaction failed for not having enough gas, try increasing the gas limit gently.
```